Generated from `docs/handoff-guides/markdown/README.md` by `tools/guide-publisher`.

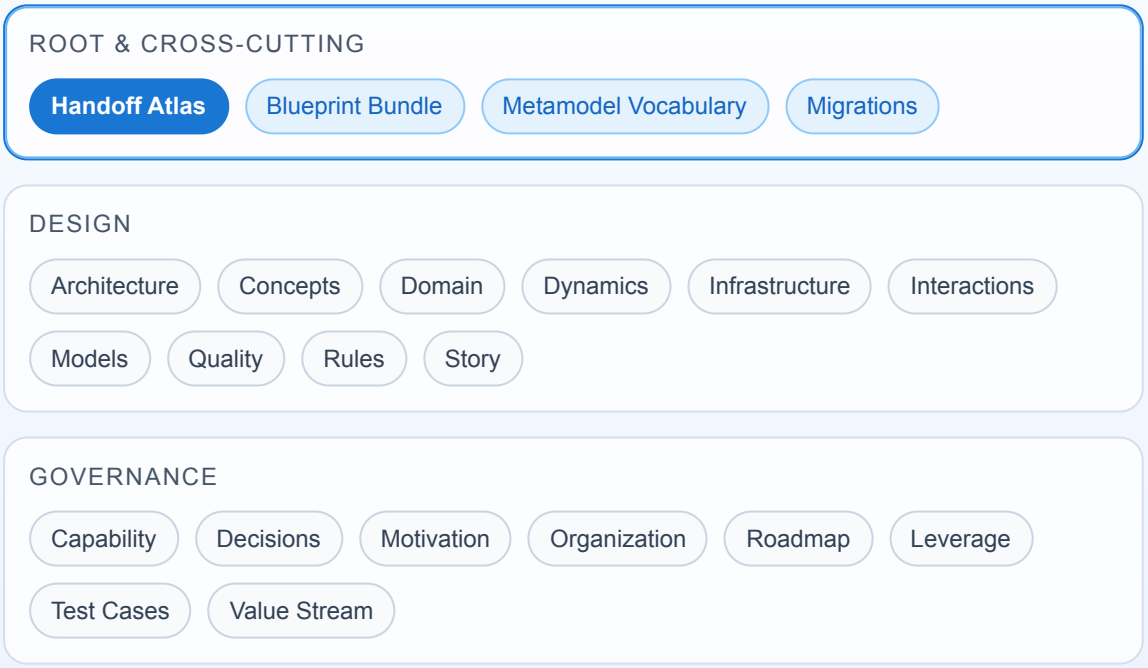
Blueprint Handoff Atlas

This guide family is for **non-technical contributors** who need to describe a business, process, screen flow, concept, risk, or decision clearly enough that a technical modeler or AI agent can turn that description into Blueprint YAML.

This is **not** a schema reference. It explains **what knowledge to capture**, **how to express it**, and **how to hand it off**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.



Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



- Cross-layer consistency

express this layer clearly.

- Shared guide map
- Review loop

One layer can span multiple files

Use thematic file splits when the knowledge is large, semantically clustered, or owned by different teams.

concepts.yaml

One file is enough when the layer is still small and coherent.

→

consumer.concepts.yaml

organization.concepts.yaml

internal.concepts.yaml

Split by meaningful theme, not arbitrary numbering. The files still describe one layer of knowledge.

What you are trying to do

Blueprint handoff starts from plain language:

- name the important **things** in the domain
- describe the **relationships** between them
- explain the **flows** and **working steps**
- capture the **screens/interactions** people use
- record **rules, goals, risks, evidence, and decisions**

If that knowledge is clear, a technical person or AI agent can transform it into YAML. If that knowledge is vague, mixed, or contradictory, the YAML will also be vague, mixed, or contradictory.

How to use this guide family

1. Start with the layer that best matches the knowledge you have.
2. Read the **Main entities in this guide** section near the top so you use the layer's actual Blueprint terms before writing free-text notes.
3. Write plain-language notes using the prompts in that guide.
4. Keep one kind of knowledge in one layer as much as possible.
5. Hand the text to a technical modeler or AI agent for YAML transformation.

6. Review the result together and correct missing or wrong meaning.

What makes good input

Good handoff input is:

- **concrete** - it names real actors, things, actions, and outcomes
- **layer-aware** - it does not mix concepts, screens, stories, and risks into one blob
- **vocabulary-aware** - it reuses the guide's named entities instead of inventing new labels
- **connected** - it explains relationships, not only isolated items
- **outcome-focused** - it explains what changes, not only what exists
- **reviewable** - another person can read it and point out gaps or contradictions

What makes a good handoff

These guides are not only for writing; they are for **handoff** between:

- business / UX / product / facilitation people, and
- technical modelers or AI agents who turn the capture into YAML

Across all guides, a good handoff should include:

- the **layer name** or guide name being used
- the **scope** of what is included and excluded
- consistent names for actors, concepts, and outcomes
- explicit links to related layers when relevant
- open questions marked clearly instead of hidden inside confident wording
- examples separated from rules or definitions

If one handoff package spans multiple files of the same type, the split should be thematic and the relationship between the files should be obvious to the receiver.

The layers and root guides at a glance

Use these guides depending on what you are trying to capture. The family has:

- **root/cross-cutting layers** - how the whole Blueprint bundle hangs together
- **design layers** - what the system is and how it works
- **governance layers** - why it exists, who owns it, how it changes, and how it is proven

Root and cross-cutting guides

Layer	Knowledge Area	Use it when you need to describe
Blueprint Bundle	Whole blueprint setup and scope.	the overall system model, its scope, layout, slices, shared files, and bundle-level principles
Metamodel Vocabulary	Shared language across the blueprint. Technically, shared semantic language and reference consistency.	cross-layer vocabulary, typed IDs, shared meanings, and reference consistency
Migrations	Planned model changes over time. Technically, model evolution and change traceability.	model change, upgrade intent, ordered changes, rollback, and impact over time

Design Layers

Layer	Knowledge Area	Use it when you need to describe
Architecture	Big-picture system structure. Technically, structural system design and decomposition.	system boundaries, parties, contexts, services, and dependencies
Concepts	Business terms and meanings. Technically, domain semantics and business ontology.	business terms, entities, identities, relationships, and state meaning
Domain	Business actions and changes. Technically, business behavior and state-change logic.	operations, events, queries, documents, errors, and key questions
Dynamics	Timing, sequencing, and coordination. Technically, runtime behavior and temporal coordination.	runtime ordering, timing, parallel work, and race conditions
Infrastructure	Operational environment and supporting technology. Technically, operational environment and platform topology.	environments, resources, topology, and operational ownership

Interactions	Screens and user interaction flow.	screens, actions, responses, navigation, and user-system exchanges
Models	Information shapes and views. Technically, information structures and data representations.	data shapes crossing boundaries or shown to users
Quality	Quality expectations and measures. Technically, quality attributes and measurable fitness.	measures, KPIs, SLOs, security, compliance, resilience, and observability
Rules	Business rules and decision criteria. Technically, business constraints and decision logic.	obligations, prohibitions, derivations, validation, and transitions
Story	Business journeys and scenarios.	journeys, processes, activities, steps, and outcomes

Governance Layers

Layer	Knowledge Area	Use it when you need to describe
Capability	What the business needs to be able to do. Technically, stable business abilities and operating model scope.	what the business can do, independent of teams or processes
Decisions	Important choices and why they were made. Technically, decision rationale and governance traceability.	explicit choices, rationale, status, and impact
Motivation	Goals, risks, assumptions, and trade-offs. Technically, strategic intent, risk, and assumption management.	goals, non-goals, risks, assumptions, trade-offs, and open questions
Organization	Teams, ownership, and responsibilities.	parties, departments, teams, and ownership
Roadmap	Delivery milestones and work planning. Technically, delivery planning and execution sequencing.	milestones, deliverables, dependencies, and success criteria

Leverage	The few priorities that will move the system forward the most. Technically, leverage points, Pareto-core intervention choices, and consequence-based prioritization.	the most important interventions, what they address, what they unlock, and what should be watched
Test Cases	Proof that behavior works as expected.	scenarios proving expected, edge, error, and fitness behavior
Value Stream	How value flows from trigger to outcome. Technically, end-to-end value delivery flow.	end-to-end value stages across capabilities and actors

How plain-language capture becomes YAML

There are two roles in the handoff:

- **non-technical contributor**
  - explains the real-world meaning
  - provides examples, context, and relationships
  - confirms whether the transformed output is faithful
- **technical modeler or AI agent**
  - maps the capture into the right Blueprint layer
  - structures the content into YAML
  - resolves references, IDs, and traceability
  - asks follow-up questions where the source text is incomplete

The goal is **faithful transformation**, not inventing meaning that was never captured.

What `blueprint.schema` means for authors

You do **not** need to study `blueprint.schema` directly.

What matters for you is this:

- it describes the overall contract of a Blueprint bundle
- it is why the final model needs enough structure and completeness
- it is why mixed-up or contradictory input creates problems later

In simple terms: it is the reason your input needs to be clear and well separated by layer.

If you need to describe the **whole Blueprint bundle** in plain language, use the dedicated **Blueprint Bundle guide**.

What `metamodel.schema` means for authors

You do **not** need to read `metamodel.schema` line by line.

What matters for you is this:

- it is the shape behind the Blueprint language
- it is why consistent naming matters
- it is why the same concept should not be described three different ways in three different guides

In simple terms: it is the reason vocabulary and relationships must stay consistent.

If you need to align people on shared **cross-layer vocabulary and typed references**, use the dedicated **Metamodel Vocabulary guide**.

What migrations mean for authors

Blueprint evolves over time. Migrations exist because the language changes.

For authors, this means:

- older notes may need updating before reuse
- examples from earlier versions may not map perfectly to the latest guide set
- keeping capture clear and plain-language makes migration easier

In simple terms: if the language evolves, clear source thinking survives better than tool-specific wording.

If you need to describe **how the model changes over time**, use the dedicated **Migrations guide**.

Cross-layer consistency checklist

Before handing off your notes, check:

- are the same **names** used consistently?
- are the same **concepts** described the same way in every guide?
- do **stories**, **interactions**, and **concepts** refer to one another clearly?
- are **actors** named consistently?

- are **outcomes**, **rules**, and **risks** stated without contradiction?

Recommended capture workflow

1. Start with rough notes from interviews, workshops, or observation.
2. Move the notes into the right layer guide.
3. Fill missing actors, relationships, steps, and outcomes.
4. Hand the result to a technical modeler or AI agent.
5. Review the transformed YAML for fidelity.
6. Correct meaning first, formatting second.

When one layer needs multiple files

Even non-technical contributors should know that one Blueprint layer does **not** always mean one physical file.

Sometimes the same schema type is split across multiple files because the knowledge is too large, has natural semantic clusters, or belongs to different ownership groups.

Good reasons to split one layer into multiple files

- the content grows beyond a manageable size
- there are clear semantic clusters
- different teams own different subsets
- you want the file layout to reveal the domain structure more clearly

Examples:

- `consumer.concepts.yaml` and `organization.concepts.yaml`
- `checkout.domain.yaml` and `fulfillment.domain.yaml`
- `pricing.rules.yaml` and `compliance.rules.yaml`
- `happy-path.story.yaml` and `error-handling.story.yaml`

What this means for non-technical authors

You should still think in terms of **one layer of knowledge**, but you may choose to organize your capture into **multiple thematic documents** inside that layer.

For example:

- one Concepts capture for customer-facing terms and another for internal/back-office terms

- one Story capture for the main journey and another for exception handling
- one Rules capture for policy rules and another for compliance rules

Good practice

- split by **meaningful theme**, not arbitrary numbering
- keep the same kind of knowledge together
- use names that explain the cluster in business language
- avoid splitting too early if one clear document is still enough
- keep cross-file vocabulary consistent so the transformed YAML still reads like one model

Important scoping rule

The same schema type can appear:

- at the **root level** for shared/system-wide concerns, and
- inside **slices** for domain-specific concerns

These are related, but they are **not the same scope**.

In plain terms:

- root files describe the forest
- slice files describe the trees

Naming pattern to know

When the technical model is split into multiple files of the same type, the common pattern is:

- `{semantic-prefix}.{schema-type}.yaml`

Examples:

- `consumer.concepts.yaml`
- `checkout.story.yaml`
- `security.decisions.yaml`

You do not need to produce final filenames yourself, but it helps to organize your source notes in the same spirit: by clear semantic cluster.

PDF exports

Generated PDFs for workshop/offline sharing live in `../pdf/`:

- Blueprint Handoff Atlas PDF
- one PDF per layer guide, generated from the Markdown sources in this folder

The PDF set also includes root/cross-cutting guides for Blueprint Bundle, Metamodel Vocabulary, and Migrations.

Common mistakes

- mixing screens, concepts, stories, and risks into one document
- describing UI layout but not user intent
- naming activities without outcomes
- naming concepts without relationships
- using examples as if they were the whole rule
- writing implementation details before business meaning is clear

Start here

Use these first:

- **Story handoff guide**
- **Interactions handoff guide**
- **Concepts handoff guide**

Then continue with the rest of the design, governance, and root guides listed above.